MVP 2.0 设计与实施文档：全链路验证原型

1. 核心目标

验证 **“后端解算 -> 中台存储/分发 -> UE4 插值渲染 -> Web 前端控制”** 的全链路可行性。 **不追求**：UI 美观、真实的物理流体、复杂的场景建模。 **追求**：数据流的低延迟、画面同步的准确性、架构的完整性。

2. 算法调研与数据协议定义 (针对后端需求)

由于后端只负责“集成”，不负责“发明”，我们需要告诉后端去找什么样的开源算法库。

2.1 人群仿真 (Crowd Simulation)

- **推荐算法库**：**RVO2 (Reciprocal Velocity Obstacles)** 或 **Social Force Model (SFM)**。
  - 原因：RVO2 是最经典的开源库（有 Python/C++ 版本），计算极快，能算出“避免碰撞的路径”。
- **后端输出数据**：RVO2 会输出每个 Agent 的 `(x, y)` 坐标和 `(vx, vy)` 速度向量。
- **动画状态判定（前端 vs 后端）**：
  - **结论**：采用**混合模式**。
  - **位置/朝向**：完全由后端数据驱动。
  - **行走/奔跑切换**：**由 UE4 前端决定**。
    - 理由：UE4 的动画状态机（AnimGraph）非常擅长根据 `Velocity` 的长度（Speed）自动混合 Idle/Walk/Run 的动画。如果让后端发“现在是跑”，UE 还要去匹配速度，容易造成“滑步”（脚在动但人没走，或者人走了脚不动）。
  - **特殊状态（跌倒/浸泡）**：**由后端决定**。
    - 理由：后端知道水深是否淹过了人的阈值。后端发一个 `State: "Drowning"`，UE4 收到后直接触发“挣扎”动画。

2.2 水灾仿真 (Flood Simulation)

- **推荐算法**：**体积填充法 (Volume Filling)** 或 **简化版浅水方程 (SWE)**。
  - 理由：对于地铁站，通常只需要知道“某个房间/区域的水位高度”。不需要像电影特效那样算每一滴水的飞溅。
- **数据需求**：`WaterHeight` (当前区域水位 Z 轴高度), `FlowSpeed` (流速，可选)。

2.3 核心数据帧结构 (JSON Protocol)

这是中台存入 MongoDB 并发给 UE 的核心结构：

```
{
  "frameId": 120,           // 第 120 帧
  "timestamp": 4.0,         // 仿真里的第 4.0 秒
  "env": {
    "waterLevel": 45.5,     // 水面高度 (UE坐标 Z轴)
    "flowSpeed": 2.0        // 流速 (用于控制材质纹理流动)
  },
  "agents": [
    {
```

```
      "id": 101,            // 角色 ID
      "pos": [1200, -500], // X, Y 坐标 (Z 由 UE 地面决定)
      "vel": [150, 0],     // 速度向量 (用于计算朝向)
      "state": 0           // 0:正常, 1:跌倒, 2:被淹
    },
    {
      "id": 102,
      "pos": [1250, -480],
      "vel": [140, 10],
      "state": 0
    }
  ]
}
```

---

3. MVP 场景与资产规格 (UE4.26)

- **场景 (Map)**：
  - 使用 **BSP Brush** 搭建一个长方形走廊（例如 20米 x 5米）。
  - 地面材质：简单的网格图（方便看移动距离）。
- **角色 (Actor)**：
  - **BP_Agent**：
    - Mesh：圆柱体 (Cylinder) 或 简单的 Mannequin (小白人)。
    - 逻辑：包含 `TargetPosition` (Vector) 和 `CurrentState` (Enum) 变量。
    - **插值逻辑**：在 `Event Tick` 中使用 `VInterpTo` 节点，将自身位置平滑过渡到 `TargetPosition`。
- **水体 (Actor)**：
  - **BP_FloodWater**：
    - Mesh：一个简单的平面 (Plane)，初始位置在地面下。
    - Material：半透明蓝色材质，加一个 **Panner** 节点连接到 Normal Map，Panner 的 Speed 参数绑定到 Material Parameter Collection，由蓝图根据后端传来的 `flowSpeed` 控制。
    - 逻辑：根据 `waterLevel` 设置 Actor 的 Z 轴高度。

---

4. 全链路交互流程 (修正版)

这里我们将你的描述转化为标准的技术时序：

第一阶段：配置与计算 (HTTP + Trigger)

1. **Web 前端**：用户在简单的 HTML 页面输入“进水量：500m³”，点击**【开始仿真】**。
2. **Web 前端** -> (HTTP POST) -> **中台 (Node.js)**：发送配置参数。
3. **中台** -> (Internal) -> **数据库**：创建 `Simulation_Task_001` 记录。
4. **中台** -> (Trigger) -> **后端 (Python/Mock)**：启动计算脚本。
   - 注：在 *MVP* 中，你可以写一个*简单*的 *Python* 脚本模*拟*后端，生成 *10* 秒的数据，每 *0.1* 秒生成一*帧*。
5. **后端** -> (TCP/HTTP) -> **中台**：高频发送计算好的 JSON 帧。
6. **中台**：收到一帧 -> 存入 MongoDB -> (可选) 推送进度给前端（如 "计算中 40%"）。
7. **中台** -> (WebSocket) -> **Web 前端**：发送消息 `{"event": "SIM_COMPLETE", "totalTime": 10}`。

第二阶段：回放与交互 (WebSocket + Pixel Streaming)

1. **Web 前端**：显示进度条（00:00 / 00:10），显示【播放】按钮。
2. **Web 前端**：
    - 自动连接 **Pixel Streaming** (WebRTC) 接收画面。
    - 自动连接 **中台 WebSocket** (用于控制数据流)。
3. **用户操作**：点击【播放】。
4. **Web 前端** -> (WebSocket) -> **中台**：发送 `{"cmd": "PLAY", "speed": 1.0}`。
5. **中台**：启动定时器，按频率（如 30ms/次）从 MongoDB 读取下一帧数据，推送到 **UE4**。
6. **UE4**：
    - 通过 WebSocket 收到 JSON。
    - 解析 `agents` 数组。
    - **关键逻辑**：遍历数组，找到对应 ID 的 `BP_Agent`，更新其 `TargetPosition`。
    - **渲染**：`Event Tick` 平滑插值移动 Actor。
    - **推流**：Pixel Streaming 插件将画面编码发回前端。
7. **Web 前端**：用户看到两个圆柱体在走廊里平滑移动，水位上涨。
8. **用户操作**：按下 `W/A/S/D`。
    - Web 前端通过 Pixel Streaming 的 `EmitUIInteraction` 发送指令。
    - UE4 的 `GameMode` 捕获指令，移动 **CameraPawn**（观察者视角），而不影响正在跑动的圆柱体。

---

5. 你的待办事项 (To-Do List)

作为负责**中台**和**UE集成**的角色，你需要做：

1. **数据生成器 (Mock Backend)**：
    - 别等后端。写一个 Node.js 脚本，生成一个 `data.json` 文件。
    - 内容：2 个 ID，前 5 秒走直线，后 5 秒加速，水位从 Z=-100 升到 Z=50。
    - 目的：*这就验证了你对*算法数据的理解。
2. **中台服务 (Node.js)**：
    - 实现 WebSocket Server。
    - 实现"读取 JSON 数组并按时间间隔发送"的逻辑（模拟播放器）。
3. **UE4 接收端 (Blueprint)**：
    - 在 4.26 中找一个好用的 WebSocket 插件（推荐 **WebSockets** 模块配合 C++ 或 插件市场免费的）。
    - 写蓝图：解析 JSON -> `Get All Actors of Class` -> Loop -> Set Target Location。
4. **插值验证**：
    - 中台设置为 1秒发 1 次数据（极低频）。
    - UE4 设置 `VInterp Speed`。
    - 观察圆柱体是否是"瞬移"还是"慢慢滑过去"。如果是慢慢滑过去，**验证成功**。